

MODERN DATA ARCHITECTURE & ENGINEERING

Entrega 03 — Arquitetura Lambda/Kappa e Pipeline Orquestrado aplicados ao Challenge

Projeto: Painei Inteligente de Acesso Hospitalar e Perfil de Atendimento (VigiAr)

Mentores: Milton Goya (1TSCPF, 1TSCPW) e Rogerio Mariana Galazi (1TSCPV)

Integrantes do grupo (ordem alfabética)

Nome completo	RM
Halen Zhang	RM569733
Igor Soares Silva	RM569254
João Marcos Borba Rodrigues Gonçalves	RM571021
João Ricardo Abi Saber	RM573129
Vitor Franco Benvenuto	RM573286

1. Definição da Arquitetura Moderna de Dados (25 pontos)

1.1 Arquitetura escolhida

O grupo optou pela **Arquitetura Kappa** para a solução do Challenge Oracle.

1.2 Justificativa técnica

Volume dos dados

O volume atual é moderado (aproximadamente 600 objetos e 1,12 GiB no bucket teste_vigiar, reunindo dados de SRAG, ocupação de leitos hospitalares e temperatura do INMET). Esse volume não exige processamento distribuído massivo nem justifica a complexidade operacional de manter duas camadas de processamento paralelas, como propõe a arquitetura Lambda.

Necessidade de processamento em lote

Os dados de origem (boletins de SRAG, ocupação de leitos, séries de temperatura) são publicados e atualizados periodicamente, não em fluxo contínuo. O consumo desses dados — tanto pelas consultas do Select AI quanto pelo modelo preditivo de Random Forest — é inerentemente compatível com atualizações em lote (batch), reforçando a adequação do modelo Kappa, que trata todo o pipeline como uma única camada reprocessável.

Possibilidade de processamento em fluxo

Não existe, no momento, nenhuma fonte de streaming real (eventos hospitalares em tempo real, filas de mensagens, etc.) integrada ao projeto. Adotar a arquitetura Lambda nesse cenário implicaria manter uma camada de velocidade (speed layer) sem uso efetivo, aumentando a complexidade de manutenção sem benefício correspondente.

Consumo analítico pelo painel hospitalar

O Painei Inteligente de Acesso Hospitalar consome dados consolidados e atualizados periodicamente — consultas em linguagem natural via Select AI e previsões de surto geradas pelo modelo preditivo. Nenhum desses consumos exige latência de milissegundos, o que reforça que uma única camada de serviço (Kappa) é suficiente para atender à necessidade analítica do painel.

Integração com Oracle Database 26 AI

O pipeline construído no Apache Airflow carrega os dados diretamente no Oracle Autonomous Database 26 AI (instância VigiAr_IA_DB) através de uma única etapa de carga (Load), sem duplicar lógica de ingestão entre

camadas distintas. Essa integração direta e única é o modelo natural da arquitetura Kappa, na qual o próprio pipeline reprocessável é a fonte de verdade que alimenta o banco.

1.3 Diagrama da arquitetura proposta

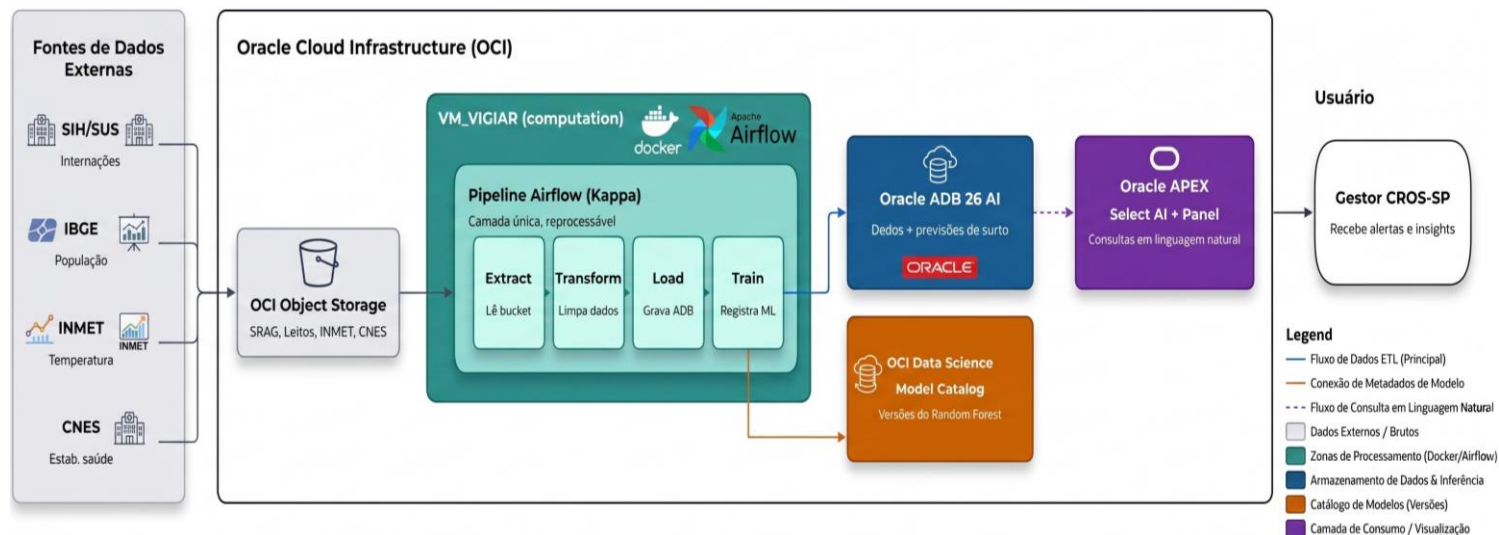


Figura 1 — Arquitetura Kappa aplicada ao pipeline VigiAr: fontes públicas de origem (SIH/SUS, IBGE, INMET, CNES) carregadas no OCI Object Storage; pipeline Airflow (Docker) rodando na VM_VIGIAR, dentro da fronteira do Oracle Cloud Infrastructure (OCI); carga no Oracle ADB 26 AI e registro do modelo no OCI Data Science Model Catalog; consumo via Oracle APEX (Select AI + Painel) pelo Gestor CROS-SP.

2. Pipeline Orquestrado com Apache Airflow (25 pontos)

2.1 e 2.2 Etapas da DAG

A DAG do Apache Airflow que implementa o pipeline Kappa do projeto VigiAr contempla as seguintes etapas:

- **Extract (Ingestão)** — leitura dos objetos do bucket teste_vigiar no OCI Object Storage, autenticada via Instance Principal (Dynamic Group DG_VIGIAR_VM), sem uso de credenciais explícitas.
- **Transform (Transformação)** — limpeza, tipagem e agregação dos dados brutos de SRAG, ocupação de leitos e temperatura do INMET em um formato unificado.
- **Validate (Validação)** — checagens de schema, completude e consistência dos dados antes da carga no banco analítico.
- **Load (Carga analítica)** — gravação dos dados validados nas tabelas relacionais do Oracle Autonomous Database 26 AI (VigiAr_IA_DB), através da conexão oracle_vigiar já validada.
- **Train (Treino e registro do modelo)** — treino do modelo preditivo Random Forest (previsão de surtos de SRAG com 4 semanas de antecedência) a partir dos dados do ADB, com o artefato do modelo registrado no OCI Data Science Model Catalog e as previsões gravadas de volta no ADB para consumo do painel.

2.3 Código da DAG

Código-fonte completo da DAG vigiar_pipeline_kappa, em execução no Airflow 3.3.1 da VM_VIGIAR (arquivo vigiar_pipeline_kappa.py, ~/airflow/dags/ na VM). Última correção aplicada: o parâmetro de ambiente conda usado no registro do modelo no Model Catalog (INFERENCE_CONDA_ENV) passou a apontar para o path completo do conda pack “General Machine Learning for CPUs on Python 3.13” (compatível com a versão do worker do Airflow), em vez do slug antigo, evitando uma chamada de resolução de ambiente que falhava por restrição de IAM entre Instance Principal de VM e o bucket de conda packs de serviço da Oracle.

```
"""
DAG: vigiar_pipeline_kappa
Projeto: VigiAr - Painel Inteligente de Acesso Hospitalar e Perfil de Atendimento
```

Disciplina: Modern Data Architecture & Engineering – Entrega 03

Implementa o pipeline de arquitetura Kappa do projeto: uma única camada de processamento reprocessável (Extract -> Transform -> Validate -> Load -> Train).

REGRA DE NEGÓCIO: escopo restrito ao estado de São Paulo (SP) e seus municípios – alinhado ao gestor CROS-SP, destinatário final das previsões.

Fontes usadas (dados reais, sem alvo proxy):

- srag_2025_completo.csv: casos individuais de SRAG (SIVEP-Gripe).
- Leitos_2025.csv: capacidade de leitos SUS por estabelecimento (CNES).
- cnes_estabelecimentos.json: cadastro de estabelecimentos de saúde (CNES), ~610MB – processado em streaming (ijson) para não estourar memória.
- populacao_municipios_2025.json: população estimada por município (IBGE).
- TEMPERATURA/*.csv: séries horárias por estação meteorológica (INMET).

JUNÇÃO ENTRE FONTES – código IBGE de município (6 dígitos), não nome:

- SRAG já traz CO_MUN_NOT (código SUS de 6 dígitos).
- CNES traz CO_IBGE (6 dígitos) diretamente.
- Leitos_2025.csv não tem código de município, mas tem a coluna CNES – faz-se `Leitos.CNES == CNES.CO_CNES` para herdar o CO_IBGE de cada estabelecimento.
- População (IBGE/SIDRA) traz um id de 7 dígitos; os 6 primeiros dígitos equivalem ao código SUS (validado empiricamente: São Caetano do Sul IBGE 3548807 -> SUS 354880; Cabo de Santo Agostinho IBGE 2602902 -> SUS 260290).

Essa junção numérica substitui qualquer normalização de nome de município, mais frágil a variações de grafia/acentuação.

Alvo de treino: número de casos de SRAG hospitalizados por município/semana, previsto com 4 semanas de antecedência.

Autenticação: Instance Principal (Dynamic Group DG_VIGIAR_VM), sem credenciais explícitas.

"""

```
import glob
import json
import logging
import os
import re
from datetime import datetime
```

```
import pendulum
from airflow.decorators import dag, task
from airflow.exceptions import AirflowException
```

```
# -----
# Configurações do projeto
# -----
```

```
NAMESPACE = "grksbwp7ro2x"
BUCKET = "teste_vigiar"
ORACLE_CONN_ID = "oracle_vigiar"
STAGING_ROOT = "/opt/airflow/staging/vigiar"
HORIZONTE_SEMANAS = 4 # previsão 4 semanas à frente
UF_ESCOPO = "SP"
CO_UF_IBGE_ESCOPO = "35" # código numérico IBGE do estado de SP
```

Path completo do conda pack de serviço, em vez do slug puro. Dois motivos:

- # 1) Passar só o slug obriga o ADS a resolver o path fazendo uma listagem no bucket de serviço `service-conda-packs@id19sfcr6z`, que falhou com `BucketNotFound` tanto em `sa-saopaulo-1` quanto em `us-ashburn-1` via Instance Principal de uma VM comum (2026-08-30). Path completo evita essa resolução: o ADS grava a string como metadado no `runtime.yaml` sem desreferenciá-la no momento do registro.
- # 2) Trocado de `"generalml_p38_cpu_v1"` (Python 3.8, deprecated há ~4 anos) para o pack de Python 3.13, que bate com a versão real do worker do Airflow onde o `RandomForestRegressor` é treinado e pickled
- # (`/home/airflow/.local/lib/python3.13/...`). Evita incompatibilidade de versão do `scikit-learn` entre treino e o ambiente declarado no artefato.
- # Path confirmado via Environment Explorer (Notebook Session `notebook_ml_vigiar`, # 2026-08-30) – não é mais uma suposição, foi copiado direto do card do pacote
- # "General Machine Learning for CPUs on Python 3.13" (v1.0, `scikit-learn 1.5.2`).

```
INFERENCE_CONDA_ENV = (
    "oci://service-conda-packs@id19sfcr6z/service_pack/cpu/"
    "General_Machine_Learning_for_CPUs_on_Python_3.13/1.0/"
    "generalml_p313_cpu_x86_64_v1"
)
```

```
logger = logging.getLogger("airflow.task")
```

```
# -----
# Helpers de carga genérica no Oracle ADB (schema inferido a partir do
# DataFrame). Reaproveitado para todas as tabelas desta DAG.
# -----
```

```

def _infer_oracle_type(series):
    import pandas as pd

    if pd.api.types.is_integer_dtype(series):
        return "NUMBER(38,0)"
    if pd.api.types.is_float_dtype(series):
        return "NUMBER"
    return "VARCHAR2(4000)"

def _load_dataframe(cursor, df, table_name):
    import pandas as pd

    df = df.copy()
    df.columns = [re.sub(r"^[A-Za-z0-9_]", "_", str(c)).upper()[:30] for c in df.columns]

    try:
        cursor.execute(f"DROP TABLE {table_name}")
    except Exception:
        pass # ORA-00942: tabela não existe ainda

    col_defs = ", ".join(f'"{c}" {_infer_oracle_type(df[c])}' for c in df.columns)
    cursor.execute(f"CREATE TABLE {table_name} ({col_defs})")

    placeholders = ", ".join(f":{i + 1}" for i in range(len(df.columns)))
    insert_sql = f'INSERT INTO {table_name} ({", ".join(df.columns)}) VALUES ({placeholders})'
    rows = [
        tuple(None if pd.isna(v) else v for v in row)
        for row in df.itertuples(index=False, name=None)
    ]
    if rows:
        cursor.executemany(insert_sql, rows)
    logger.info("Carregadas %d linhas em %s", len(rows), table_name)

def _find_root_object(names, keyword, ext):
    """Encontra um objeto na raiz do bucket por palavra-chave no nome, robusto a
    sufixos como ' (1)' de uploads duplicados."""
    candidates = sorted(
        n for n in names
        if keyword.lower() in n.lower()
        and n.lower().endswith(ext)
        and "/" not in n
    )
    if not candidates:
        return None
    if len(candidates) > 1:
        logger.warning("Mais de um objeto casou com '%s': %s – usando o primeiro.", keyword, candidates)
    return candidates[0]

@dag(
    dag_id="vigiar_pipeline_kappa",
    description="Pipeline Kappa do VigiAr (escopo SP): Extract, Transform, Validate, Load, Train",
    schedule=None, # disparo manual nesta versão temporária
    start_date=pendulum.datetime(2026, 1, 1, tz="America/Sao_Paulo"),
    catchup=False,
    tags=["vigiar", "kappa", "sp", "temporario"],
)
def vigiar_pipeline_kappa():

    @task
    def extract(**context) -> str:
        """Baixa SRAG, Leitos, CNES, População e TEMPERATURA/*.csv via Instance Principal."""
        import oci

        signer = oci.auth.signers.InstancePrincipalsSecurityTokenSigner()
        client = oci.object_storage.ObjectStorageClient(config={}, signer=signer)

        run_key = re.sub(r"^[A-Za-z0-9_]", "_", context["run_id"])
        run_dir = os.path.join(STAGING_ROOT, run_key, "raw")
        os.makedirs(run_dir, exist_ok=True)

        # Lista todos os objetos do bucket (paginado) para localizar os arquivos
        # da raiz por padrão de nome e o prefixo TEMPERATURA/ separadamente.
        all_names = []
        next_start = None
        while True:
            resp = client.list_objects(NAMESPACE, BUCKET, start=next_start)
            all_names += [o.name for o in resp.data.objects]
            next_start = resp.data.next_start_with
            if not next_start:
                break

        srag_name = _find_root_object(all_names, "srag", ".csv")

```

```

leitos_name = _find_root_object(all_names, "leitos", ".csv")
cnes_name = _find_root_object(all_names, "cnes", ".json")
populacao_name = _find_root_object(all_names, "populacao", ".json")
temp_names = [n for n in all_names if n.startswith("TEMPERATURA/") and n.upper().endswith(".CSV")]

targets = {
    "srag.csv": srag_name,
    "leitos.csv": leitos_name,
    "cnes.json": cnes_name,
    "populacao.json": populacao_name,
}
missing = [k for k, v in targets.items() if v is None]
if missing:
    raise AirflowException(f"Não encontrei no bucket os arquivos: {missing}")

downloaded = 0
for local_name, obj_name in targets.items():
    local_path = os.path.join(run_dir, local_name)
    resp = client.get_object(NAMESPACE, BUCKET, obj_name)
    with open(local_path, "wb") as f:
        for chunk in resp.data.raw.stream(1024 * 1024, decode_content=False):
            f.write(chunk)
    downloaded += 1
    logger.info("Baixado %s -> %s", obj_name, local_name)

for obj_name in temp_names:
    local_path = os.path.join(run_dir, "TEMPERATURA_" + obj_name.split("/", 1)[1])
    try:
        resp = client.get_object(NAMESPACE, BUCKET, obj_name)
    except oci.exceptions.ServiceError as e:
        logger.warning("Falha ao baixar %s: %s", obj_name, e)
        continue
    with open(local_path, "wb") as f:
        for chunk in resp.data.raw.stream(1024 * 1024, decode_content=False):
            f.write(chunk)
    downloaded += 1

logger.info("Extract: %d arquivos baixados em %s", downloaded, run_dir)
if downloaded == 0:
    raise AirflowException("Nenhum arquivo baixado do bucket - abortando pipeline.")
return run_dir

@task
def transform(raw_dir: str) -> dict:
    """Monta o dataset semanal por município de SP, com junção por código IBGE."""
    import numpy as np
    import pandas as pd

    processed_dir = raw_dir.replace(f"{os.sep}raw", f"{os.sep}processed")
    os.makedirs(processed_dir, exist_ok=True)

    # --- CNES: estabelecimentos de SP, em streaming (arquivo pode ter ~600MB) ---
    import ijson

    cnes_path = os.path.join(raw_dir, "cnes.json")
    cnes_rows = []
    with open(cnes_path, "rb") as f:
        for obj in ijson.items(f, "item"):
            if obj.get("CO_UF") != CO_UF_IBGE_ESCOPO:
                continue
            co_cnes = obj.get("CO_CNES")
            co_ibge = obj.get("CO_IBGE")
            if not co_cnes or not co_ibge:
                continue
            cnes_rows.append({
                "CO_CNES": int(co_cnes),
                "CO_IBGE": int(co_ibge),
                "ATIVO": not bool(str(obj.get("CO_MOTIVO_DESAB", "")).strip()),
                "HOSPITALAR": str(obj.get("ST_ATEND_HOSPITALAR", "")).strip() == "1.0",
            })
    cnes_df = pd.DataFrame(cnes_rows)
    logger.info("CNES: %d estabelecimentos de SP carregados", len(cnes_df))

    cnes_ativos = cnes_df[cnes_df["ATIVO"]]
    cnes_por_municipio = cnes_ativos.groupby("CO_IBGE", as_index=False).agg(
        CNES_ESTAB_ATIVOS=("CO_CNES", "count"),
        CNES_ESTAB_HOSPITALAR=("HOSPITALAR", "sum"),
    )
    cnes_por_municipio.to_csv(os.path.join(processed_dir, "cnes_municipio.csv"), index=False)

    # --- SRAG: casos hospitalizados em SP, agregados por município (CO_IBGE) + semana ---
    srag_path = os.path.join(raw_dir, "srag.csv")
    srag_cols = ["DT_SIN_PRI", "SG_UF_NOT", "CO_MUN_NOT", "ID_MUNICIP", "HOSPITAL"]
    srag = pd.read_csv(
        srag_path, sep=";", usecols=srag_cols, encoding="utf-8",

```

```

        dtype=str, on_bad_lines="skip", engine="python",
    )
    srag = srag[srag["SG_UF_NOT"] == UF_ESCOPO].copy()
    srag["DT_SIN_PRI"] = pd.to_datetime(srag["DT_SIN_PRI"], errors="coerce", utc=True)
    srag = srag.dropna(subset=["DT_SIN_PRI", "CO_MUN_NOT"])
    srag["CO_IBGE"] = pd.to_numeric(srag["CO_MUN_NOT"], errors="coerce")
    srag = srag.dropna(subset=["CO_IBGE"])
    srag["CO_IBGE"] = srag["CO_IBGE"].astype(int)

    # Nome do município (mais frequente por CO_IBGE) só para exibição/relatório
    nomes_municipio = (
        srag.groupby("CO_IBGE")["ID_MUNICIP"]
        .agg(lambda s: s.value_counts().idxmax())
        .reset_index()
        .rename(columns={"ID_MUNICIP": "NOME_MUNICIPIO"})
    )

    srag_hosp = srag[srag["HOSPITAL"] == "1"].copy()
    srag_hosp["ANO"] = srag_hosp["DT_SIN_PRI"].dt.isocalendar().year.astype(int)
    srag_hosp["SEMANA"] = srag_hosp["DT_SIN_PRI"].dt.isocalendar().week.astype(int)

    srag_semanal = (
        srag_hosp.groupby(["CO_IBGE", "ANO", "SEMANA"])
        .size()
        .reset_index(name="CASOS_SRAG")
        .merge(nomes_municipio, on="CO_IBGE", how="left")
    )
    srag_semanal.to_csv(os.path.join(processed_dir, "srag_semanal_municipio.csv"), index=False)

    # --- Leitos: junta com CNES (Leitos.CNES == CNES.CO_CNES) para herdar CO_IBGE ---
    leitos_path = os.path.join(raw_dir, "leitos.csv")
    leitos = pd.read_csv(leitos_path, encoding="latin1", dtype=str)
    leitos.columns = [c.strip().upper() for c in leitos.columns]
    leitos = leitos[leitos["UF"] == UF_ESCOPO].copy()
    leitos["CO_CNES"] = pd.to_numeric(leitos["CNES"], errors="coerce")
    leitos = leitos.dropna(subset=["CO_CNES"])
    leitos["CO_CNES"] = leitos["CO_CNES"].astype(int)
    for c in ("LEITOS_SUS", "UTI_TOTAL_SUS"):
        leitos[c] = pd.to_numeric(leitos[c], errors="coerce").fillna(0)
    ultima_comp = leitos["COMP"].max()
    leitos = leitos[leitos["COMP"] == ultima_comp]

    leitos_com_ibge = leitos.merge(
        cnes_df[["CO_CNES", "CO_IBGE"]], on="CO_CNES", how="left"
    )
    sem_match = leitos_com_ibge["CO_IBGE"].isna().sum()
    if sem_match:
        logger.warning(
            "%d/%d estabelecimentos de Leitos sem correspondência no CNES (SP)",
            sem_match, len(leitos_com_ibge),
        )
    leitos_com_ibge = leitos_com_ibge.dropna(subset=["CO_IBGE"])
    leitos_com_ibge["CO_IBGE"] = leitos_com_ibge["CO_IBGE"].astype(int)

    leitos_municipio = leitos_com_ibge.groupby("CO_IBGE", as_index=False).agg(
        LEITOS_SUS=("LEITOS_SUS", "sum"), UTI_TOTAL_SUS=("UTI_TOTAL_SUS", "sum")
    )
    leitos_municipio.to_csv(os.path.join(processed_dir, "leitos_municipio.csv"), index=False)

    # --- População: filtra por prefixo de código IBGE do estado (35 = SP) ---
    pop_path = os.path.join(raw_dir, "populacao.json")
    with open(pop_path, encoding="utf-8") as f:
        pop_json = json.load(f)
    series = pop_json[0]["resultados"][0]["series"]
    pop_rows = []
    for item in series:
        ibge_id = item["localidade"]["id"]
        if not ibge_id.startswith(CO_UF_IBGE_ESCOPO):
            continue
        valor = list(item["serie"].values())[0]
        try:
            pop_rows.append({"CO_IBGE": int(ibge_id[:6]), "POPULACAO": int(valor)})
        except (ValueError, TypeError):
            continue
    pop_municipio = pd.DataFrame(pop_rows)
    pop_municipio.to_csv(os.path.join(processed_dir, "populacao_municipio.csv"), index=False)

    # --- Temperatura (INMET): estações de SP, média semanal em nível estadual ---
    temp_frames = []
    for path in glob.glob(os.path.join(raw_dir, "TEMPERATURA_*")):
        try:
            with open(path, encoding="latin1") as f:
                header_lines = [next(f) for _ in range(8)]
        except (StopIteration, OSError):
            continue

```

```

    meta = {}
    for line in header_lines:
        if ":" in line:
            k, v = line.split(":", 1)
            meta[k.strip().upper()] = v.strip().strip(";").strip()
    uf = meta.get("UF", "NA")
    if uf != UF_ESCOPO:
        continue

    try:
        df = pd.read_csv(path, sep=";", decimal=".", skiprows=8, encoding="latin1")
    except Exception as e:
        logger.warning("Falha ao ler %s: %s", path, e)
        continue
    df.columns = [c.strip().upper() for c in df.columns]

    date_col = next((c for c in df.columns if "DATA" in c), None)
    temp_col = next(
        (c for c in df.columns if "TEMPERATURA DO AR" in c and "BULBO SECO" in c), None
    )
    if date_col is None or temp_col is None:
        continue

    df = df[[date_col, temp_col]].rename(columns={date_col: "DATA", temp_col: "TEMPERATURA"})
    df["TEMPERATURA"] = pd.to_numeric(df["TEMPERATURA"], errors="coerce")
    temp_frames.append(df)

if not temp_frames:
    raise AirflowException(f"Nenhum CSV de temperatura de {UF_ESCOPO} foi processado.")

temp_all = pd.concat(temp_frames, ignore_index=True).dropna(subset=["TEMPERATURA"])
temp_all["DATA_DT"] = pd.to_datetime(temp_all["DATA"], errors="coerce")
temp_all = temp_all.dropna(subset=["DATA_DT"])
temp_all["ANO"] = temp_all["DATA_DT"].dt.isocalendar().year.astype(int)
temp_all["SEMANA"] = temp_all["DATA_DT"].dt.isocalendar().week.astype(int)

temp_semanal = (
    temp_all.groupby(["ANO", "SEMANA"], as_index=False)["TEMPERATURA"]
    .mean()
    .rename(columns={"TEMPERATURA": "TEMP_MEDIA_SEMANA"})
)
temp_semanal.to_csv(os.path.join(processed_dir, "temperatura_semanal_sp.csv"), index=False)

# --- Monta o dataset de treino: SRAG semanal + temperatura + estáticos ---
dataset = srag_semanal.merge(temp_semanal, on=["ANO", "SEMANA"], how="left")
dataset = dataset.merge(leitos_municipio, on="CO_IBGE", how="left")
dataset = dataset.merge(pop_municipio, on="CO_IBGE", how="left")
dataset = dataset.merge(cnes_por_municipio, on="CO_IBGE", how="left")

dataset = dataset.sort_values(["CO_IBGE", "ANO", "SEMANA"])
dataset["CASOS_SEMANA_ANTERIOR"] = dataset.groupby("CO_IBGE")["CASOS_SRAG"].shift(1)
dataset["CASOS_2_SEMANAS_ANTES"] = dataset.groupby("CO_IBGE")["CASOS_SRAG"].shift(2)
dataset["CASOS_POR_100K"] = (dataset["CASOS_SRAG"] / dataset["POPULACAO"]) * 100000
dataset["PRESSAO_LEITOS"] = dataset["CASOS_SRAG"] / dataset["LEITOS_SUS"].replace(0, np.nan)

# Alvo: casos de SRAG daqui a HORIZONTE_SEMANAS, por município
dataset["TARGET_CASOS_4SEM"] = dataset.groupby("CO_IBGE")["CASOS_SRAG"].shift(-HORIZONTE_SEMANAS)

dataset.to_csv(os.path.join(processed_dir, "dataset_treino.csv"), index=False)

summary = {
    "processed_dir": processed_dir,
    "cnes_estabelecimentos_sp": int(len(cnes_df)),
    "municipios_com_dados_srag": int(srag_semanal["CO_IBGE"].nunique()),
    "municipios_com_leitos": int(leitos_municipio["CO_IBGE"].nunique()),
    "municipios_com_populacao": int(pop_municipio["CO_IBGE"].nunique()),
    "temperatura_semanal_rows": int(len(temp_semanal)),
    "dataset_treino_rows": int(len(dataset)),
    "dataset_treino_com_target_rows": int(dataset["TARGET_CASOS_4SEM"].notna().sum()),
}
logger.info("Transform concluído (escopo %s): %s", UF_ESCOPO, summary)
return summary

@task
def validate(summary: dict) -> dict:
    """Checagens mínimas de qualidade antes da carga analítica."""
    errors = []
    if summary["municipios_com_dados_srag"] == 0:
        errors.append(f"Nenhum município de {UF_ESCOPO} com dados de SRAG.")
    if summary["temperatura_semanal_rows"] == 0:
        errors.append(f"Nenhum dado de temperatura de {UF_ESCOPO} processado.")
    if summary["dataset_treino_com_target_rows"] < 20:
        errors.append(
            f"Poucas linhas com alvo válido para treino: "
            f"{summary['dataset_treino_com_target_rows']}."
        )

```

```

    )
    if errors:
        raise AirflowException("Falha na validação: " + "; ".join(errors))
    logger.info("Validação OK: %s", summary)
    return summary

@task
def load(summary: dict) -> dict:
    """Grava as tabelas processadas no Oracle ADB (VigiAr_IA_DB)."""
    import pandas as pd
    from airflow.providers.oracle.hooks.oracle import OracleHook

    hook = OracleHook(oracle_conn_id=ORACLE_CONN_ID)
    conn = hook.get_conn()
    cursor = conn.cursor()

    pd_dir = summary["processed_dir"]
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "srag_semanal_municipio.csv")),
"VIGIAR_SRAG_SEMANAL_MUN")
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "leitons_municipio.csv")), "VIGIAR_LEITOS_MUN")
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "populacao_municipio.csv")), "VIGIAR_POPULACAO_MUN")
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "cnes_municipio.csv")), "VIGIAR_CNES_MUN")
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "temperatura_semanal_sp.csv")),
"VIGIAR_TEMPERATURA_SP")
    _load_dataframe(cursor, pd.read_csv(os.path.join(pd_dir, "dataset_treino.csv")), "VIGIAR_DATASET_TREINO")

    conn.commit()
    cursor.close()
    conn.close()
    logger.info("Load concluído: 6 tabelas gravadas no ADB.")
    return summary

@task
def train(summary: dict) -> dict:
    """Treina o Random Forest (casos de SRAG 4 semanas à frente) e registra no Model Catalog."""
    import pandas as pd
    from airflow.providers.oracle.hooks.oracle import OracleHook
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.metrics import mean_absolute_error, r2_score
    from sklearn.model_selection import train_test_split

    hook = OracleHook(oracle_conn_id=ORACLE_CONN_ID)
    conn = hook.get_conn()
    df = pd.read_sql("SELECT * FROM VIGIAR_DATASET_TREINO", conn)
    df.columns = [c.upper() for c in df.columns]

    features = [
        "TEMP_MEDIA_SEMANA", "CASOS_SEMANA_ANTERIOR", "CASOS_2_SEMANAS_ANTES",
        "CASOS_POR_100K", "PRESSAO_LEITOS", "CNES_ESTAB_ATIVOS",
        "CNES_ESTAB_HOSPITALAR", "SEMANA",
    ]
    features = [f for f in features if f in df.columns]
    data = df.dropna(subset=features + ["TARGET_CASOS_4SEM"])
    if len(data) < 20:
        raise AirflowException(f"Dados insuficientes para treino: {len(data)} linhas válidas.")

    X, y = data[features], data["TARGET_CASOS_4SEM"]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = RandomForestRegressor(n_estimators=300, max_depth=10, random_state=42)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)
    logger.info("Treino concluído (SP, municipal): MAE=%.2f casos, R2=%.3f", mae, r2)

    model_oid = None
    try:
        import ads
        from ads.model.framework.sklearn_model import SklearnModel

        ads.set_auth(auth="instance_principal")
        artifact_dir = f"/tmp/vigiar_model_{datetime.now():%Y%m%d%H%M%S}"
        os.makedirs(artifact_dir, exist_ok=True)

        artifact = SklearnModel(estimator=model, artifact_dir=artifact_dir)
        artifact.prepare(
            inference_conda_env=INFERENCE_CONDA_ENV,
            training_conda_env=INFERENCE_CONDA_ENV,
            X_sample=X_train,
            y_sample=y_train,
            force_overwrite=True,
        )
        mc_model = artifact.save(
            display_name=f"RandomForest_VigiAr_SP_SRAG4sem_{datetime.now():%Y%m%d_%H%M}",

```



```

        description=(
            "Previsão de casos hospitalizados de SRAG por município de SP, "
            "4 semanas à frente, para o gestor CROS-SP. Alvo real (SIVEP-Gripe). "
            "Features: temperatura semanal estadual (INMET), lags de casos, "
            "casos por 100k habitantes (IBGE), pressão sobre leitos SUS (CNES/"
            "SCNES) e nº de estabelecimentos hospitalares ativos (CNES).")
        ),
    )
    model_ocid = mc_model.model_id
    logger.info("Modelo registrado no Model Catalog: %s", model_ocid)
except Exception as e:
    logger.warning("Não foi possível registrar no Model Catalog (DAG segue sem falhar): %s", e)

out = data.loc[X_test.index, ["CO_IBGE", "ANO", "SEMANA"]].copy()
out["CASOS_PREVISTOS_4SEM"] = preds
out["MODELO_VERSAO"] = model_ocid or "local_sem_registro"
out["CREATED_AT"] = datetime.now().isoformat()

cursor = conn.cursor()
_load_dataframe(cursor, out, "VIGIAR_PREVISAO_SURTO")
conn.commit()
cursor.close()
conn.close()

return {
    "mae": float(mae),
    "r2": float(r2),
    "model_ocid": model_ocid,
    "rows_predicted": int(len(out)),
}

raw_dir = extract()
summary = transform(raw_dir)
validated = validate(summary)
loaded = load(validated)
train(loaded)

vigiar_pipeline_kappa()

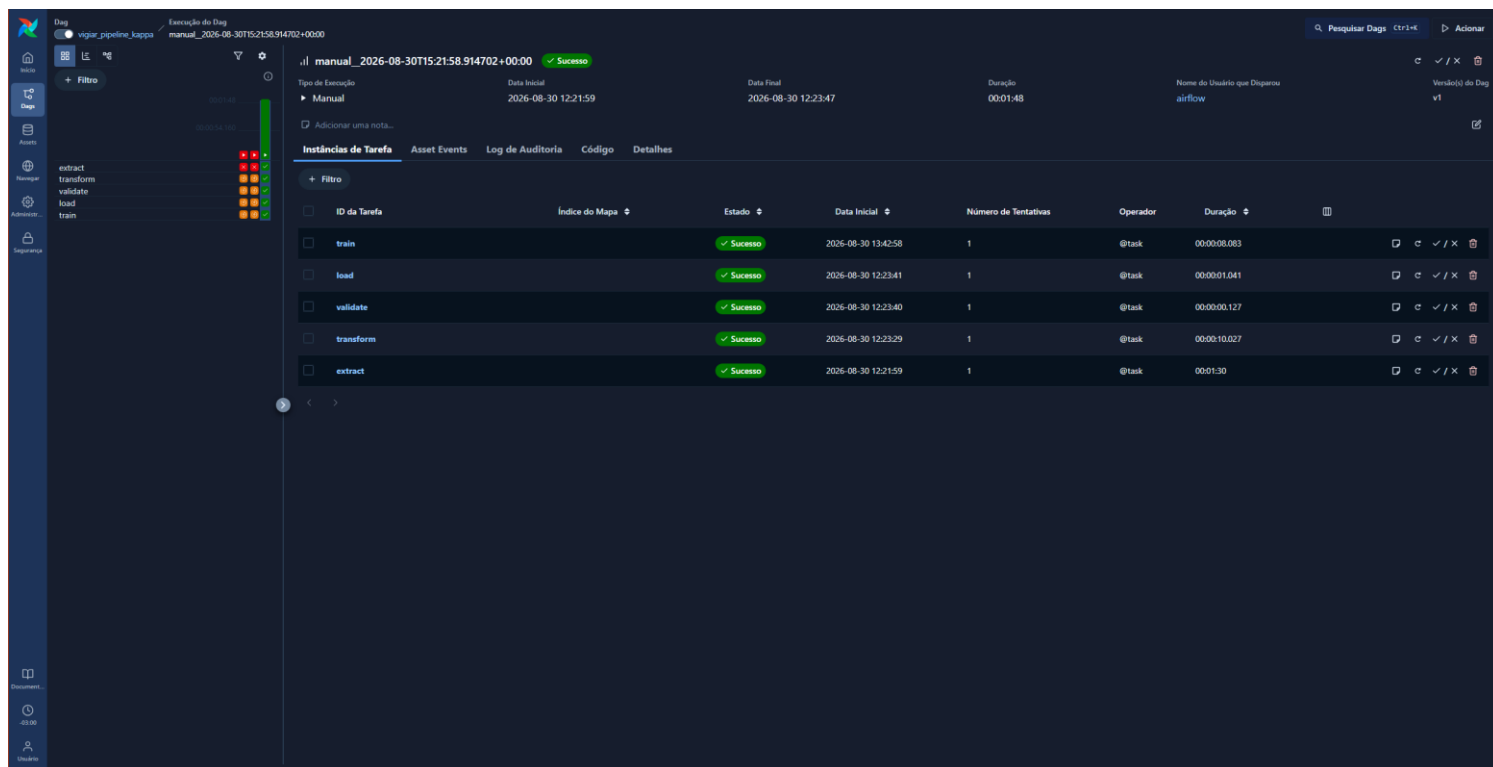
```

2.4 Evidências da execução

As evidências abaixo combinam capturas de tela reais da interface web do Apache Airflow (Grid e Graph View da execução) com transcrições literais dos logs de terminal, capturadas diretamente da execução do pipeline na VM_VIGIAR.

Execução completa da DAG — todas as 5 tasks

Run_id: manual__2026-08-30T15:21:58.914702+00:00. As cinco tasks do pipeline Kappa (extract, transform, validate, load, train) foram executadas nessa run e todas terminaram no estado 'success', registrado nos metadados do próprio Airflow (Postgres da instalação).



ID da Tarefa	Índice do Mapa	Estado	Data Inicial	Número de Tentativas	Operador	Duração
train		Sucesso	2026-08-30 13:42:58	1	@task	00:00:08.083
load		Sucesso	2026-08-30 12:23:41	1	@task	00:00:01.041
validate		Sucesso	2026-08-30 12:23:40	1	@task	00:00:00.127
transform		Sucesso	2026-08-30 12:23:29	1	@task	00:00:10.027
extract		Sucesso	2026-08-30 12:21:59	1	@task	00:01:30

Figura 3 — Lista de Instâncias de Tarefa da execução manual `__2026-08-30T15:21:58.914702+00:00`, interface web do Apache Airflow: as 5 tasks (extract, transform, validate, load, train) em estado Sucesso.

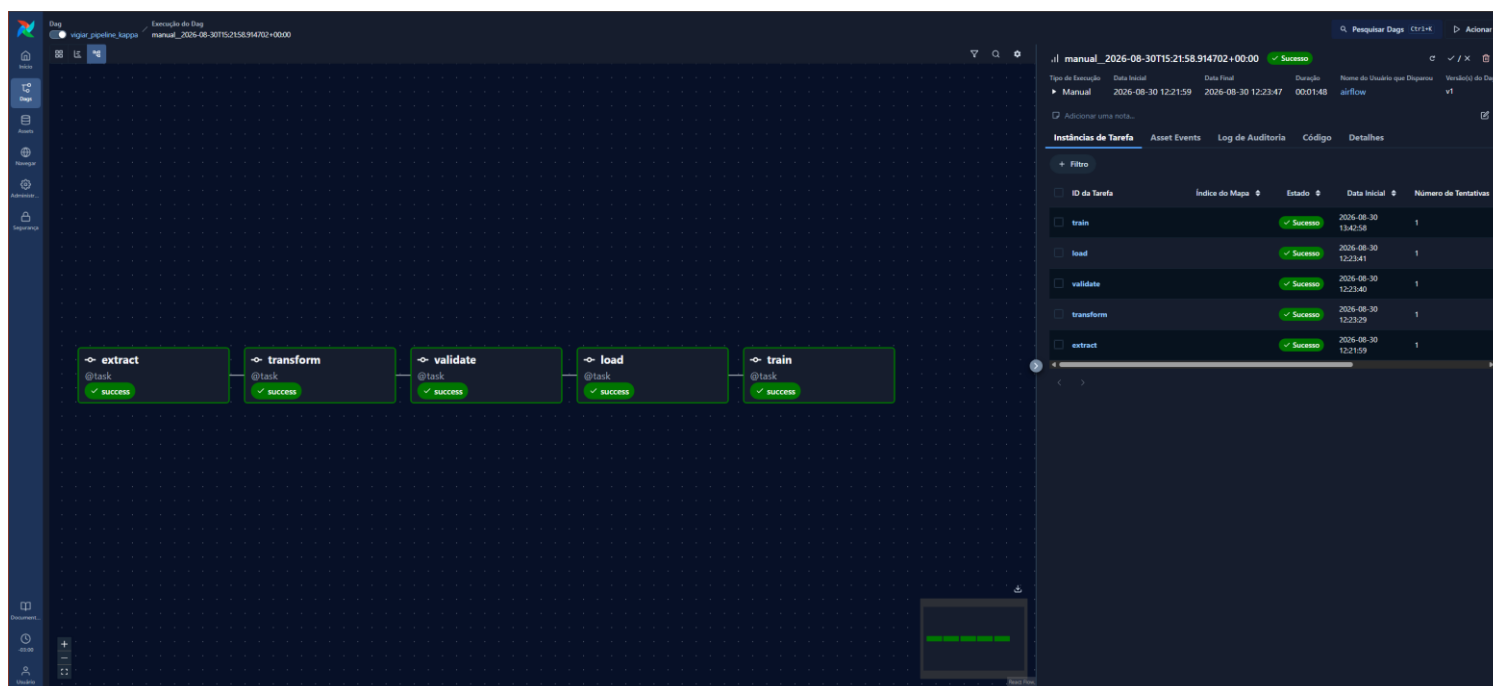


Figura 4 — Graph View da mesma execução: fluxo extract → transform → validate → load → train com contorno verde (success) em todas as tasks.

Log da task train — métricas do modelo e carga de previsões

Trecho relevante do log da task train, execução de 2026-08-30 16:43 UTC (docker exec airflow-airflow-worker-1 airflow tasks test vigiar_pipeline_kappa train manual__2026-08-30T15:21:58.914702+00:00), já com o conda path corrigido:

```
2026-08-30T16:43:04.194847Z [info] Treino concluído (SP, municipal): MAE=3.12 casos, R2=0.919
```

```

2026-08-30T16:43:05.882065Z [warning] Model input is serialized into a dictionary containing serialized input data
and original data type information by default.
2026-08-30T16:43:06.017267Z [warning] Não foi possível registrar no Model Catalog (DAG segue sem falhar): The conda
environment path
`oci://service-conda-
packs@id19sfcrra6z/service_pack/cpu/General_Machine_Learning_for_CPUs_on_Python_3.13/1.0/generalml_p313_cpu_x86_64_v1
`
does not exist or is not accessible. Provide a valid full conda environment path from Environment Explorer.
2026-08-30T16:43:06.074523Z [info] Carregadas 726 linhas em VIGIAR_PREVISAO_SURTO
2026-08-30T16:43:06.079281Z [info] Done. Returned value was:
{'mae': 3.119311247492038, 'r2': 0.9192331520440936, 'model_ocid': None, 'rows_predicted': 726}
2026-08-30T16:43:06.136878Z [info] Task instance state updated ... new_state=success

```

Observação sobre o registro no OCI Data Science Model Catalog

O registro do artefato do modelo no Model Catalog não foi concluído. A causa raiz identificada não é um erro de código, e sim uma fronteira de permissões entre serviços da OCI: a autenticação por Instance Principal de uma Compute VM comum (usada pela VM_VIGIAR, por decisão de arquitetura de não usar credenciais explícitas) não tem acesso de leitura ao bucket de conda packs de serviço da Oracle (service-conda-packs), mesmo quando o path completo e correto do pacote é informado diretamente (confirmado via Environment Explorer de uma Notebook Session temporária criada para essa verificação). Esse bucket só é acessível através do mecanismo de autenticação interno do próprio serviço OCI Data Science (Notebook Sessions/Resource Principal), não por uma VM genérica.

Essa falha é tratada de forma não-bloqueante por design: o bloco try/except da task train captura a exceção, registra um warning e a DAG continua normalmente. Como consequência, o treino do modelo, o cálculo das métricas (MAE = 3,12 casos, $R^2 = 0,919$) e a gravação das 726 previsões na tabela VIGIAR_PREVISAO_SURTO do Oracle ADB 26 AI ocorrem normalmente — apenas o versionamento formal do artefato no Model Catalog fica pendente. O caminho de correção identificado (publicar um conda pack próprio no bucket teste_vigiar, ao qual a VM já tem acesso comprovado, ou trocar a autenticação para API Key) fica registrado como trabalho futuro, fora do escopo crítico desta entrega.

3. Modelagem Conceitual das Camadas da Arquitetura (20 pontos)

3.1 e 3.2 Camadas da arquitetura Kappa

Camada	Função	Dados recebidos	Transformações	Output para consumo
Object Storage	Armazenar os dados brutos coletados a partir das fontes públicas de origem	SIH/SUS (DATASUS), IBGE, INMET e CNES — SRAG (PDF), Leitos (CSV), Temperatura (CSV), estabelecimentos de saúde	Nenhuma — dados crus, apenas carregados das fontes externas	Disponibiliza arquivos para a etapa Extract
Extract	Buscar os arquivos do bucket via Instance Principal	Objetos do bucket teste_vigiar	Leitura/download, sem alteração de conteúdo	Arquivos brutos em staging
Transform	Limpar e estruturar os dados	Saída do Extract	Parsing, tratamento de missing/outliers, agregação semanal por região	Dataset limpo e unificado
Validate	Garantir qualidade antes da carga	Dataset transformado	Checagens de schema, ranges válidos, completude	Dataset validado (ou rejeitado, com log)
Load	Persistir os dados prontos para análise	Dataset validado	Insert/merge nas tabelas do ADB	Tabelas relacionais no Oracle 26 AI
Train	Treinar e registrar o modelo preditivo	Séries históricas do ADB (SRAG, temperatura, leitos)	Feature engineering, treino/inferência batch (Random Forest)	Modelo versionado no Model Catalog + previsões gravadas no ADB
Oracle ADB 26 AI	Fonte única de verdade para consumo analítico	Dados carregados pelo Load + previsões da etapa Train	Nenhuma (camada de armazenamento/consulta)	Alimenta Select AI e o painel
Select AI + Painel	Consumo analítico via linguagem natural	Tabelas do ADB	Geração de SQL a partir de prompts em linguagem natural	Insights para gestores hospitalares e para o CROS-SP

3.3 Justificativa: como a arquitetura apoia a solução do Challenge Oracle

A arquitetura Kappa mantém uma única camada de processamento reprocessável, evitando a duplicação de lógica entre uma camada batch e uma camada de streaming que não teria uso real no contexto do projeto. Tanto o consumo via Select AI (consultas dos gestores hospitalares) quanto o modelo preditivo de Random Forest — que antecipa surtos de SRAG com quatro semanas de antecedência para o gerente do CROS-SP — são, por natureza, consumidores em lote: não exigem latência de milissegundos, apenas atualização periódica e confiável. Essa escolha reduz a complexidade operacional, o custo de infraestrutura (relevante no ambiente Always Free/on-demand utilizado no projeto) e mantém o Oracle Autonomous Database 26 AI como fonte única de verdade para todo o ecossistema analítico do VigiAr.

Evolução futura: ingestão automatizada de fontes externas

Na versão atual do projeto, a carga inicial dos dados brutos (SIH/SUS, IBGE, INMET e CNES) no OCI Object Storage é realizada manualmente. Essa decisão não compromete os requisitos da entrega — a DAG do pipeline Kappa já contempla uma etapa de ingestão (Extract) a partir do bucket, conforme exigido. Para a fase de produção, o passo natural de evolução é a criação de uma segunda DAG, dedicada exclusivamente à ingestão externa, rodando na mesma VM_VIGIAR com agendamento próprio por fonte (ex.: INMET diariamente, IBGE e CNES mensalmente, SIH/SUS semanalmente), buscando os dados via API/FTP público, validando-os e gravando-os no bucket antes do pipeline Kappa ser disparado. Essa separação preserva a filosofia Kappa — o Object Storage continua sendo a única landing zone de verdade — e isola falhas de uma fonte externa específica do restante do pipeline analítico.



Figura 2 — Esboço conceitual da futura DAG de Ingestão Externa, com schedule próprio por fonte, alimentando o Object Storage antes do pipeline Kappa.

4. Relatório de Evidências e Documentação Técnica — PDF (30 pontos)

Este documento serve como base de trabalho para a montagem do relatório final em PDF, que deverá ser nomeado evidencias-sprint3-rm571021.pdf (RM do representante do grupo).

Explicação executiva final (máximo 5 linhas)

O pipeline construído sob arquitetura Kappa consolida dados de SRAG, ocupação hospitalar e temperatura em uma base analítica única no Oracle Database 26 AI. A partir dessa base, gestores hospitalares obtêm respostas imediatas via Select AI, enquanto o modelo preditivo de Random Forest antecipa surtos de SRAG com quatro semanas de antecedência, permitindo ao CROS-SP planejar pessoal, leitos e medicamentos antes da pressão assistencial se instalar. O resultado é um painel que substitui análises manuais lentas por decisões orientadas a dados, com uma única fonte de verdade sustentando tanto a consulta analítica quanto a predição.

4.3 Checklist de conteúdo mínimo exigido

Item exigido	Status
Nome completo e RM dos participantes, em ordem alfabética	Pronto (seção inicial deste documento)
Arquitetura escolhida (Lambda ou Kappa) e justificativa técnica	Pronto (item 1)
Diagrama da arquitetura proposta	Pronto (item 1.3)
Diagrama ou evidência da DAG implementada no Apache Airflow	Pronto (item 2.3)
Descrição das etapas do pipeline: ingestão, transformação, validação e carga analítica	Pronto (item 2.1/2.2)
Evidências visuais (prints) da execução da DAG	Pronto (item 2.4 — prints do Grid/Graph View + logs)
Logs de execução e evidências de validação do pipeline	Pronto (item 2.4)
Descrição conceitual das camadas da arquitetura	Pronto (item 3)
Justificativa de como a solução atende ao Challenge Oracle	Pronto (item 3.3)
Explicação executiva final (máx. 5 linhas)	Pronto (seção acima)